Quantum Fourier Transform

$$F_n|j\rangle = \frac{1}{2^n} \sum_k e^{(2\pi i jk/n)} |k\rangle$$

Presented by:

KASAK(252IS018) — Part I: Discrete Fourier Transform & QFT Theory

ABHAY(252IS001) — Part II: QFT Circuits & Implementation

Outline

Part I

1. Primitive Roots of Unity
2. DFT Matrix Definition
3. Properties of DFT
4. Orthogonality (Case j=k & j≠k)
5. Convolution & Multiplication
6. Example: Transform $|\psi\rangle$ using F_4
7. Period/Wavelength Relationship

Part II

1. QFT in Product Form
2. Quantum Circuit for n-qubits
3. Rotation Gate R_k
4. 3-Qubit QFT Circuit
5. Correctness Verification
6. Swap Gates & Circuit Order
7. QFT vs Classical DFT
8. Inverse QFT

Part I

Discrete Fourier Transform & QFT Theory

Primitive Roots of Unity

Definition

$\omega^n = e^{(2\pi i/n)}$

Primitive root of unity ($z = 1$, $z \in \mathbb{C}$)

n roots: $\{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$

$|\omega^n| = \sqrt{(\omega^n \cdot \omega^{n*})} = 1$

$\omega^{nn} = \omega^n \bmod n = 1$

DFT Matrix F_n (n×n)

$(F_n)_{jk} = \omega^{n(jk)} / \sqrt{n}$

$$F_n = \begin{matrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{(n-1)} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{(n-1)} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)^2} \end{matrix}$$

(multiply by $1/\sqrt{n}$ for normalisation)

Properties of DFT

1. Unitary

$$\langle c_j \mid c_k \rangle = \delta_{jk}$$

Columns are orthonormal. F_n is a unitary matrix.

2. Orthogonality

$$\Sigma_m \, \omega^{(m(j-k))} = n \cdot \delta_{jk}$$

Case j=k: sum = n (geometric series = 1)
Case j≠k: sum = 0 (roots cancel)

3. Convolution ↔ Multiplication

$$F_n(\alpha * \beta) = F_n(\alpha) \cdot F_n(\beta)$$

Convolution in time = pointwise multiply in frequency

4. Period / Wavelength

$$f : \{0,1\}^n \to \{0,1\}^n, \quad \text{QFT } f(x) = f(x)$$

Periodic ⇒ N/r = 2^n/r (period r divides 2^n)

Worked Example: $F_4|\psi\rangle$

Transform $|\psi\rangle = |0\rangle + |3\rangle$ using F_4 $(\omega = i,\ \omega^2 = -1,\ \omega^3 = -i)$

Step 1: F_4 matrix

$$F_4 = \tfrac{1}{2} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{bmatrix}$$

$$= \tfrac{1}{2} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Step 2: Apply to $(|0\rangle + |3\rangle)$

$$= \tfrac{1}{2} \cdot \begin{bmatrix} 1+1 \\ 1-i \\ 1-1 \\ 1+i \end{bmatrix} = \tfrac{1}{2} \begin{bmatrix} 2 \\ 1-i \\ 0 \\ 1+i \end{bmatrix}$$

$$\phi = 2|0\rangle + (1-i)|1\rangle + (1+i)|3\rangle$$

$$\mathrm{Prob}(|0\rangle) = (2/2\sqrt{2})^2$$

$$\mathrm{Prob}(|1\rangle) = |(1-i)/2\sqrt{2}|^2$$

$$\mathrm{Prob}(|2\rangle) = |(1+i)/2\sqrt{2}|^2$$

Part II

QFT Circuit & Implementation

QFT in Product Form

Key Formula

For $j = j_1 j_2 \ldots j_n$ (binary):

$$
\begin{aligned}
&QFT|j\rangle = \\
&\tfrac{1}{2^n}\,[|0\rangle + e^{(2\pi i\ 0.j_n)}|1\rangle] \\
&\quad \otimes [|0\rangle + e^{(2\pi i\ 0.j_{n-1}j_n)}|1\rangle] \\
&\quad \otimes \ldots \\
&\quad \otimes [|0\rangle + e^{(2\pi i\ 0.j_1 j_2 \ldots j_n)}|1\rangle]
\end{aligned}
$$

Each qubit contributes exactly ONE tensor factor — enabling efficient circuit decomposition!

Rotation Gate R_k

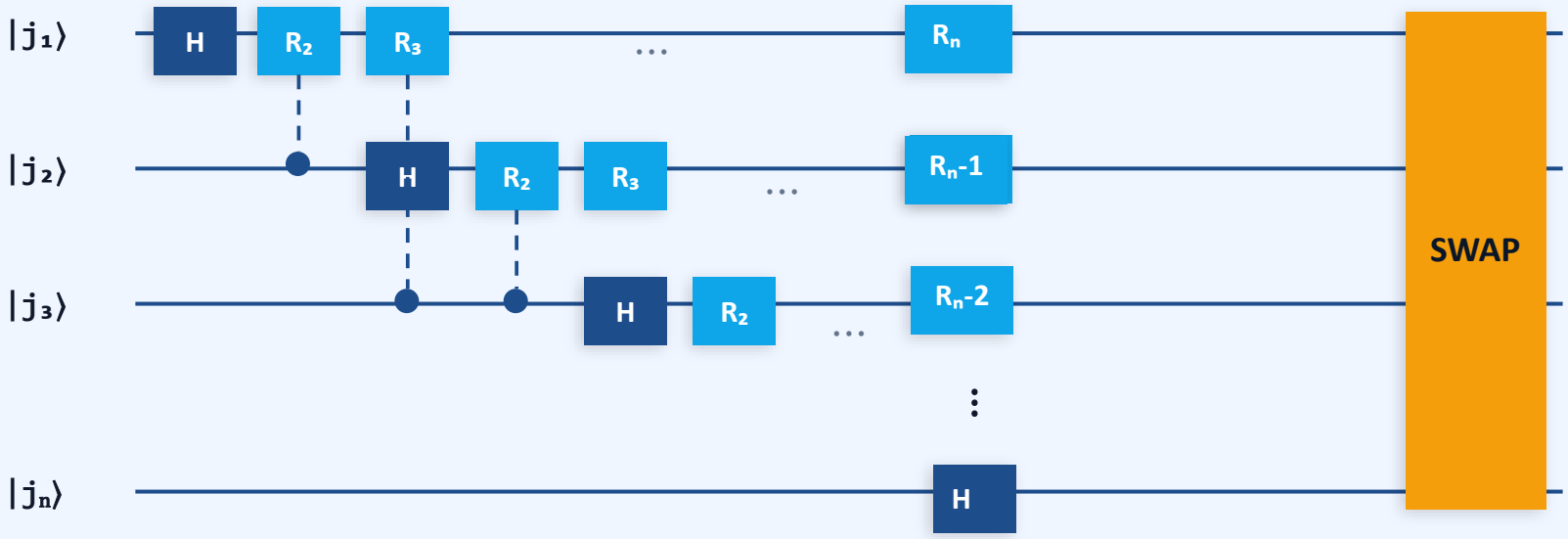
$$
R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{(2\pi i/2^k)} \end{bmatrix}
$$

$$
\begin{aligned}
R_k|0\rangle &= |0\rangle \\
R_k|1\rangle &= e^{(2\pi i/2^k)}|1\rangle
\end{aligned}
$$

Phase gate: applies a fractional phase shift

Controlled by later qubits. Larger k = smaller rotation angle.

Quantum Circuit for n-Qubit QFT

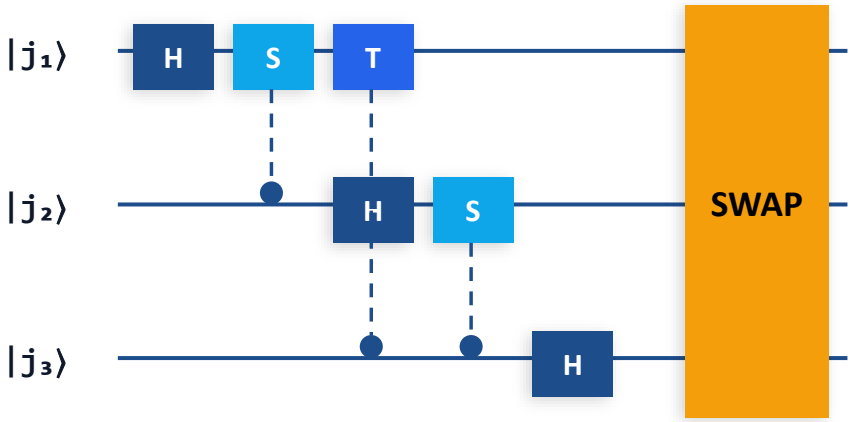


Circuit uses **$O(n^2)$** gates vs classical **$O(n \cdot \log 2^n)$** — **Exponential speedup!**

Final step: SWAP gates reverse qubit order (product form outputs in reversed bit order)

3-Qubit QFT Circuit

Circuit (n = 3)



$S = R_2$, $T = R_3$

Correctness Verification

Apply R_2 to $|j_1\rangle$:

$\frac{1}{2}(|0\rangle + e^{(2\pi i\ S_1/2)}|1\rangle)$

Apply R_n gates:

$= \frac{1}{2}(|0\rangle + e^{(2\pi i(S_1/2+S_2/4+\ldots))}|1\rangle)$

$= \frac{1}{2}(|0\rangle + e^{(2\pi i \cdot 0.j_1 j_2 \ldots j_n)}|1\rangle)$

Is it Hadamard?

$j=0$: $= \frac{1}{2}\sqrt{2}(|0\rangle+|1\rangle) = H|0\rangle$ ✓

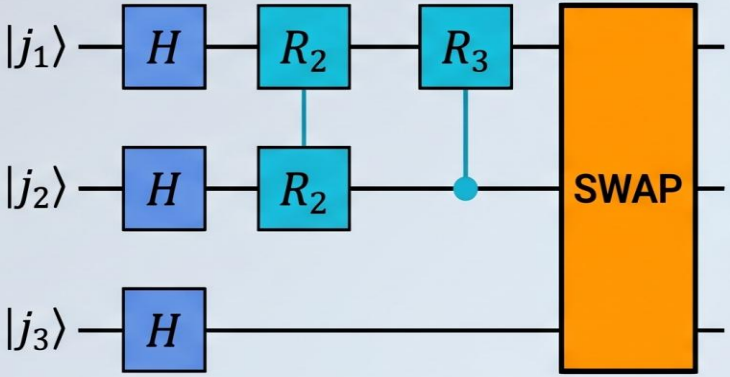
$j=1$: $= \frac{1}{2}\sqrt{2}(|0\rangle-|1\rangle) = H|1\rangle$ ✓

QFT vs Classical DFT

Property	Classical DFT	Quantum QFT
Complexity	$O(n \cdot 2^n)$	**$O(n^2)$**
Input	Classical vector $x \in \mathbb{C}^N$	**Quantum state $\lvert j \rangle$**
Output	Classical vector $\hat{f} \in \mathbb{C}^N$	**Quantum state $QFT \lvert j \rangle$**
Readout	Direct access to all coefficients	**Only via measurement (collapses state)**
Speed	FFT: $O(N \log N)$ for $N=2^n$	**$O(n^2)$ gates exponentially fewer**
Application	Signal processing, image compression	**Shor's algorithm, phase estimation**

Inverse Quantum Fourier Transform (IQFT) — The ‘Undo’ Operation

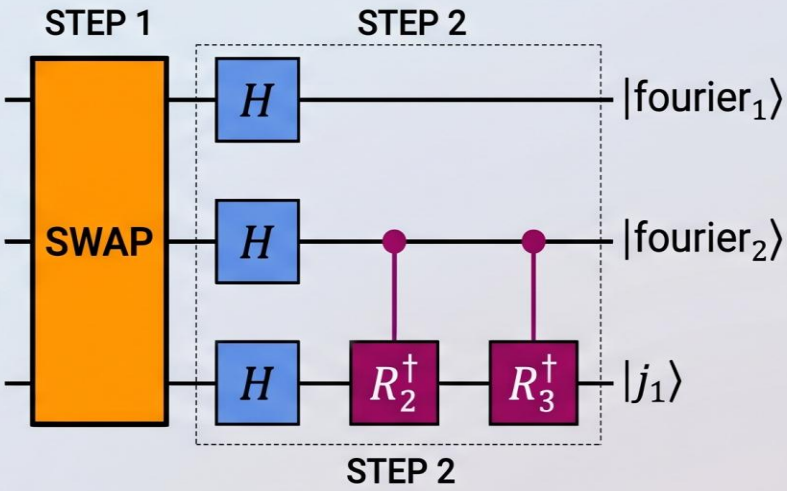
1. Standard QFT Circuit (Review)



Output in Fourier Basis (Reversed Order)

$F_n \leftrightarrow F_n^\dagger$
(Unitary Inverse)

2. Inverse QFT Circuit ($F_n^\dagger$)



Output in Computational Basis (Decoded Phase)

*Note: **Controlled-**$R_k^\dagger$ gate applies a negative phase $e^{-2\pi i/2^k}$.

Summary

DFT	Applies $\omega^n = e^{(2\pi i/n)}$ — primitive root of unity — to transform signal vectors
Properties	Unitary, orthogonal columns, convolution ↔ multiplication in frequency domain
QFT Product Form	Decomposes n-qubit transform into n independent tensor factors
Circuit	H + controlled-R_k gates per qubit + swap — only $O(n^2)$ total gates
Speedup	QFT is exponentially faster than classical FFT ($O(n^2)$ vs $O(n \cdot 2^n)$)
Applications	Foundation for Shor's factoring algorithm and quantum phase estimation

References

- Introduction to classical and Quantum Computing by Thomas G. Wong
- Quantum computing for everyone by Bernhardt
- Google Nano Banana for Image Generation

😊 THANK YOU 😊